

WORDPRESS INSTALLATION & CONFIGURATION LAB REPORT

Red Hat Linux | Apache | PHP | MariaDB | AWS EC2

Lab Environment

Platform: Amazon Web Services (AWS) EC2
OS: Red Hat Enterprise Linux (RHEL/Amazon Linux 2023)
Hostname: wordpress-server
Date: May 21, 2026

1. Objective

The objective of this lab is to install and configure a fully functional WordPress website on a Red Hat Linux server hosted on AWS EC2, using the LAMP stack — Apache as the web server, PHP as the scripting language, and MariaDB as the database server.

By the end of this lab, the following should be achieved:

- A working Apache web server serving content on port 80
- PHP 8.3 installed with all required extensions for WordPress
- MariaDB 10.11 running with a dedicated WordPress database and user
- WordPress fully installed, configured, and accessible via browser
- Basic security hardening applied including SELinux, firewall, and file permissions
- Backup procedures documented for files and database

2. Lab Tasks Overview

The following 18 tasks were assigned for this lab:

#	Task	Status
1	Configure system hostname and verify network connectivity	Completed
2	Update OS packages and verify repository availability	Completed
3	Install Apache, PHP, and MariaDB	Completed
4	Enable and start all required services	Completed
5	Configure firewall rules to allow web traffic	Completed
6	Secure the MariaDB installation	Completed
7	Create WordPress database and user with privileges	Completed
8	Download latest WordPress package from official source	Completed
9	Extract and place WordPress files in web root	Completed
10	Configure file ownership, permissions, SELinux contexts	Completed
11	Create and configure WordPress configuration file	Completed
12	Configure Apache Virtual Host for WordPress	Completed
13	Test Apache configuration and restart web service	Completed
14	Access WordPress installation page and complete setup	Completed
15	Verify database connectivity, web service, and dashboard	Completed
16	Implement basic hardening	Completed
17	Create backup of files and database	Completed
18	Document packages, services, database details, access URL	Completed

3. Lab Environment & Architecture

3.1 Server Environment

Parameter	Value
Cloud Provider	Amazon Web Services (AWS)
Instance Type	EC2 (Amazon Linux 2023 / RHEL-based)
Hostname	wordpress-server
User	ec2-user
Public IP	13.232.39.20
Web Port	80 (HTTP)
DB Port	3306 (MariaDB — localhost only)
Web Root	/var/www/html/wordpress
OS Package Manager	dnf (RPM-based)

3.2 Stack Architecture

The LAMP stack was installed and configured as follows:

Architecture Flow
Browser (Client)
↓ HTTP Request (Port 80)
AWS Security Group (Inbound: port 80 open)
↓
Apache HTTP Server (httpd) — Listens on port 80
↓ PHP-FPM processes .php files
PHP 8.3 (with extensions: pdo, mysqlnd, xml, gd, mbstring, intl, zip, opcache)
↓ DB queries via mysqli/PDO
MariaDB 10.11 — Database: wordpressdb | User: wpuser@localhost
↓
WordPress Files at /var/www/html/wordpress

3.3 AWS Security Group Configuration

The EC2 instance security group was configured to allow inbound traffic on the following ports:

Port / Protocol	Purpose
22 (SSH)	Remote server management
80 (HTTP)	WordPress web traffic
443 (HTTPS)	SSL (for future use)

4. Step-by-Step Installation

4.1 Configure System Hostname

The first step was to set the hostname of the server so that it is identifiable on the network.

```
# Set hostname
sudo hostnamectl set-hostname wordpress-server

# Verify hostname
hostnamectl

# Verify network connectivity
ping -c 4 google.com
ip addr show
```

Explanation

hostnamectl set-hostname: Sets a persistent hostname that survives reboots.

ping: Tests internet connectivity from the server.

ip addr show: Displays network interfaces and assigned IP addresses.

4.2 Update OS Packages

Before installing any packages, the system repositories were updated to ensure the latest versions are installed.

```
# Update all packages
sudo dnf update -y

# Verify repository availability
sudo dnf repolist
```

Explanation

dnf update -y: Downloads and installs all available package updates.

dnf repolist: Lists all enabled repositories to confirm availability.

On Amazon Linux 2023, the default repos include amazon-linux, extras, and epel-style repos.

4.3 Install Apache Web Server

```
# Install Apache HTTP Server
sudo dnf install -y httpd

# Verify installation
httpd -v
```

What is Apache?

Apache HTTP Server (httpd) is the world's most widely used open-source web server. It listens for HTTP requests on port 80 (and HTTPS on 443) and serves web content. In the LAMP stack, Apache handles all incoming browser requests and passes PHP files to PHP-FPM for processing. Configuration files are stored at `/etc/httpd/conf/` and virtual hosts at `/etc/httpd/conf.d/`.

4.4 Install PHP and Required Extensions

WordPress requires PHP 7.4 or higher. PHP 8.3 was installed along with all necessary extensions.

```
# Install PHP 8.3 and all required extensions for WordPress
sudo dnf install -y php php-cli php-common php-fpm php-mysqlnd php-xml \
  php-mbstring php-gd php-pdo php-opcache php-intl php-pecl-zip
```

PHP Package	Purpose in WordPress
php	Core PHP interpreter
php-fpm	FastCGI Process Manager — handles PHP requests for Apache
php-mysqlnd	MySQL Native Driver — enables database connectivity
php-xml	XML parsing — required for feeds, sitemaps
php-mbstring	Multi-byte string functions — for internationalization
php-gd	Image manipulation — thumbnails, image uploads
php-pdo	PHP Data Objects — database abstraction layer
php-opcache	Bytecode caching — speeds up PHP execution
php-intl	Internationalization functions
php-pecl-zip	ZIP archive support — plugin/theme installation

4.5 Install MariaDB Database Server

```
# Install MariaDB server and client
sudo dnf install -y mariadb105-server

# Verify MariaDB version
mysql --version
```

What is MariaDB?

MariaDB is a community-developed, open-source relational database management system (RDBMS). It is a drop-in replacement for MySQL, created by the original MySQL developers after Oracle acquired MySQL. WordPress uses MariaDB/MySQL to store all content: posts, pages, users, settings, comments, and plugin data.

On this server, MariaDB 10.11.15 was installed.

4.6 Enable and Start All Services

All three services (Apache, MariaDB, PHP-FPM) must be started and enabled to auto-start on system reboot.

```
# Enable and start Apache
sudo systemctl enable httpd
sudo systemctl start httpd

# Enable and start MariaDB
sudo systemctl enable mariadb
sudo systemctl start mariadb

# Enable and start PHP-FPM
sudo systemctl enable php-fpm
sudo systemctl start php-fpm

# Verify all services are running
sudo systemctl status httpd
sudo systemctl status mariadb
sudo systemctl status php-fpm
```

Service Status Confirmed

httpd.service — Active: active (running) since Thu 2026-05-21 09:47:14 UTC
mariadb.service — Active: active (running) since Thu 2026-05-21 06:03:50 UTC
php-fpm.service — Active: active (running) since Thu 2026-05-21 09:34:56 UTC

What does 'enable' do vs 'start'?

systemctl start SERVICE: Starts the service immediately (until next reboot).
systemctl enable SERVICE: Configures the service to start automatically every time the system boots.
Both commands together ensure the service runs now AND after every restart.

4.7 Configure Firewall Rules

```
# Check firewall status
sudo systemctl status firewalld

# If firewalld is running, allow HTTP and HTTPS
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --permanent --add-service=https
sudo firewall-cmd --reload

# Verify open ports
sudo firewall-cmd --list-all
```

Firewall Explanation

Linux systems use firewalld (or iptables) to control inbound and outbound network traffic.

--permanent: Makes the rule persistent across reboots.

--add-service=http: Opens port 80 for HTTP traffic.

--add-service=https: Opens port 443 for HTTPS traffic.

--reload: Applies the new rules without restarting the firewall daemon.

Note: On AWS EC2, traffic must also be allowed through the AWS Security Group (inbound rules).

4.8 Secure the MariaDB Installation

```
# Run the security script
sudo mysql_secure_installation

# This script will prompt you to:
# 1. Set root password
# 2. Remove anonymous users
# 3. Disallow root login remotely
# 4. Remove test database
# 5. Reload privilege tables
```

Why Secure MariaDB?

A fresh MariaDB installation has several insecure defaults:

- Anonymous users who can log in without a password
- A 'test' database accessible to everyone
- Root login allowed from any host

mysql_secure_installation removes all these risks in one interactive script.

4.9 Create WordPress Database and User

```
# Log into MariaDB as root
sudo mysql -u root -p

# Inside the MariaDB prompt:
CREATE DATABASE wordpressdb CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

CREATE USER 'wpuser'@'localhost' IDENTIFIED BY 'StrongPassword123!';

GRANT ALL PRIVILEGES ON wordpressdb.* TO 'wpuser'@'localhost';

FLUSH PRIVILEGES;

SHOW DATABASES;

SELECT User, Host FROM mysql.user;

EXIT;
```

DB Detail	Value
Database Name	wordpressdb
Database User	wpuser
Host	localhost
Character Set	utf8mb4
Collation	utf8mb4_unicode_ci

Why utf8mb4?

utf8mb4 is the full 4-byte UTF-8 character set, which supports all Unicode characters including emojis. The older 'utf8' in MySQL/MariaDB only supports 3-byte characters and can cause data loss with certain characters.

WordPress recommends utf8mb4 for full internationalization support.

4.10 Download WordPress

```
# Navigate to a working directory
cd /tmp

# Download the latest WordPress
# Note: wget was not available; curl was used instead
curl -O https://wordpress.org/latest.tar.gz

# Verify the download
ls -lh latest.tar.gz
```

Issue Encountered: wget not found

Problem: The wget command was not installed on this Amazon Linux instance.

Command attempted: wget https://wordpress.org/latest.tar.gz

Error: -bash: wget: command not found

Fix: Used curl instead, which is available by default:

curl -O https://wordpress.org/latest.tar.gz

Alternatively: sudo dnf install -y wget (to install wget)

4.11 Extract and Deploy WordPress Files

```
# Extract the downloaded archive
tar -xzf /tmp/latest.tar.gz -C /tmp/

# Copy WordPress to the web root
```

```
sudo cp -r /tmp/wordpress /var/www/html/
```

```
# Verify the files are in place
ls /var/www/html/wordpress
```

tar flags explained

- x: Extract files from the archive
- z: Decompress using gzip (.tar.gz format)
- f: Specifies the filename of the archive
- C: Specifies the destination directory for extraction

4.12 Configure File Ownership, Permissions, and SELinux

```
# Set Apache as the owner of WordPress files
sudo chown -R apache:apache /var/www/html/wordpress

# Set correct directory and file permissions
sudo find /var/www/html/wordpress -type d -exec chmod 755 {} \;
sudo find /var/www/html/wordpress -type f -exec chmod 644 {} \;

# Configure SELinux contexts (if SELinux is enforcing)
sudo chcon -t httpd_sys_rw_content_t /var/www/html/wordpress -R

# Or use restorecon to apply default contexts
sudo restorecon -R /var/www/html/wordpress

# Allow Apache to write to the filesystem (needed for plugin installs)
sudo setsebool -P httpd_unified 1
sudo setsebool -P httpd_can_network_connect 1
```

Permission Setting	Explanation
755 (directories)	Owner can read/write/execute; group and others can read/execute
644 (files)	Owner can read/write; group and others can read only
chown apache:apache	Apache process runs as the 'apache' user — must own the files
chcon httpd_sys_rw_content_t	SELinux context allowing Apache to read and write to this path
setsebool httpd_unified	Allows Apache to read/write all httpd content
setsebool httpd_can_network_connect	Allows WordPress to make outbound HTTP connections (plugin updates)

What is SELinux?

SELinux (Security-Enhanced Linux) is a mandatory access control (MAC) security mechanism built into the Linux kernel.

Even if file permissions allow access, SELinux can block it if the security context does not match.

On Red Hat/Amazon Linux, SELinux is enabled in Enforcing mode by default. Apache can only serve files with the correct SELinux context (`httpd_sys_content_t` for read-only, `httpd_sys_rw_content_t` for read-write access). Incorrect contexts are one of the most common causes of 403 Forbidden errors in RHEL-based systems.

4.13 Create WordPress Configuration File

```
# Copy the sample config
sudo cp /var/www/html/wordpress/wp-config-sample.php \
/var/www/html/wordpress/wp-config.php

# Edit the config file
sudo nano /var/www/html/wordpress/wp-config.php

# Update the following values inside the file:
define( 'DB_NAME',      'wordpressdb' );
define( 'DB_USER',      'wpuser' );
define( 'DB_PASSWORD', 'StrongPassword123!' );
define( 'DB_HOST',      'localhost' );
define( 'DB_CHARSET',   'utf8mb4' );
```

wp-config.php Explanation

`wp-config.php` is the main configuration file for WordPress.

It contains database credentials, security keys, table prefix, and debug settings.

This file must never be publicly accessible — it should be protected by file permissions and Apache rules.

The `wp-config-sample.php` is a template provided with WordPress that is copied and then edited.

4.14 Configure Apache Virtual Host

```
# Create a new virtual host configuration file
sudo nano /etc/httpd/conf.d/wordpress.conf

# Add the following configuration:
<VirtualHost *:80>
    ServerName 13.232.39.20
    DocumentRoot /var/www/html/wordpress

    <Directory /var/www/html/wordpress>
        Options -Indexes +FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog /var/log/httpd/wordpress-error.log
    CustomLog /var/log/httpd/wordpress-access.log combined
</VirtualHost>
```

Directive	Explanation
VirtualHost *:80	Listens on all interfaces on port 80
ServerName	The domain or IP the virtual host responds to
DocumentRoot	The directory where WordPress files are located
Options -Indexes	Disables directory listing (security hardening)
+FollowSymLinks	Allows Apache to follow symbolic links
AllowOverride All	Allows .htaccess files to override Apache settings
Require all granted	Allows all visitors to access the site
ErrorLog / CustomLog	Log files for debugging and access tracking

4.15 Test Apache and Restart

```
# Test Apache configuration for syntax errors
sudo apachectl configtest

# Expected output:
# Syntax OK

# Restart Apache to apply changes
sudo systemctl restart httpd
```

Why test before restart?

apachectl configtest loads and validates the Apache configuration without actually restarting. If there is a syntax error in any .conf file, it prints the error and exits safely. Restarting Apache with a bad config file would take the web server down — testing first prevents this.

5. WordPress Web Setup

After completing the server-side configuration, the WordPress installation wizard was accessed through the browser.

5.1 Accessing the Installation Page

```
# Open in browser:  
http://13.232.39.20/  
  
# WordPress will redirect to the setup wizard:  
http://13.232.39.20/wp-admin/install.php
```

5.2 Setup Wizard Steps

The WordPress installation wizard was completed with the following details:

Field	Value Entered
Site Title	WordPress Lab
Username	admin
Password	(secure password set)
Email	(admin email set)
Search Engine Visibility	Allow search engines to index

5.3 Installation Result

WordPress Installed Successfully

WordPress was installed and the dashboard was accessible.

Site URL: <http://13.232.39.20/>

Admin URL: <http://13.232.39.20/wp-admin/>

Default 'Hello World!' post was visible on the blog page.

Admin bar visible at the top confirming logged-in state as 'admin'.

6. Verification

6.1 Installed Packages Verification

The following command was used to verify that all required packages were installed:

```
rpm -qa | grep httpd
rpm -qa | grep php
rpm -qa | grep mariadb
```

Confirmed installed packages from the terminal output:

Package	Version
httpd	2.4.63-13.el10.x86_64
httpd-core	2.4.63-13.el10.x86_64
httpd-tools	2.4.63-13.el10.x86_64
httpd filesystem	2.4.63-13.el10.noarch
redhat-logos-httpd	100.3-2.el10.noarch
php	8.3.29-1.el10_1.x86_64
php-cli	8.3.29-1.el10_1.x86_64
php-common	8.3.29-1.el10_1.x86_64
php-fpm	8.3.29-1.el10_1.x86_64
php-mysqlnd	8.3.29-1.el10_1.x86_64
php-xml	8.3.29-1.el10_1.x86_64
php-gd	8.3.29-1.el10_1.x86_64
php-mbstring	8.3.29-1.el10_1.x86_64
php-pdo	8.3.29-1.el10_1.x86_64
php-opcache	8.3.29-1.el10_1.x86_64
php-intl	8.3.29-1.el10_1.x86_64
php-pecl-zip	1.22.3-5.el10.x86_64
mariadb	10.11.15-2.el10_1.x86_64
mariadb-server	10.11.15-2.el10_1.x86_64
mariadb-server-utils	10.11.15-2.el10_1.x86_64
mariadb-common	10.11.15-2.el10_1.noarch
mariadb-errmsg	10.11.15-2.el10_1.noarch
mariadb-connector-c	3.4.4-1.el10.x86_64
mariadb-backup	10.11.15-2.el10_1.x86_64

6.2 Service Status Verification

```
sudo systemctl status httpd
sudo systemctl status mariadb
sudo systemctl status php-fpm
```

All Services Running

httpd.service: Active (running) — PID 24177 — Started Thu 2026-05-21 09:47:14 UTC
mariadb.service: Active (running) — PID 17401 — Started Thu 2026-05-21 06:03:50 UTC
php-fpm.service: Active (running) — PID 23851 — Started Thu 2026-05-21 09:34:56 UTC

6.3 Database Verification

The following queries were run inside MariaDB to confirm correct setup:

```
sudo mysql -u root -p

MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database                |
+-----+
| information_schema      |
| mysql                   |
| performance_schema      |
| sys                     |
| wordpressdb             |
+-----+
5 rows in set (0.000 sec)

MariaDB [(none)]> SELECT User, Host FROM mysql.user;
+-----+-----+
| User      | Host      |
+-----+-----+
| mariadb.sys | localhost |
| mysql      | localhost |
| root       | localhost |
| wpuser     | localhost |
+-----+-----+
4 rows in set (0.001 sec)
```

6.4 Web Access Verification

```
# Test Apache responds correctly
curl -I http://13.232.39.20/

# Expected output:
HTTP/1.1 200 OK
Server: Apache/2.4.63
Content-Type: text/html; charset=UTF-8
```

WordPress Dashboard Verified

Browser URL: <http://13.232.39.20/>

Site title 'WordPress Lab' visible in browser tab

Default 'Hello world!' blog post displayed on home page

Admin bar visible with 'Edit Site' and 'New' options

Logged in as: admin

WordPress version: Latest (as of May 2026)

7. Troubleshooting — Problems Encountered & Solutions

During this lab, several issues were encountered. This section documents each problem, the root cause analysis, and the exact fix applied.

Issue 1: wget: command not found

Attribute	Detail
Problem	wget was not available on the Amazon Linux 2023 instance
Command that failed	wget https://wordpress.org/latest.tar.gz
Error message	-bash: wget: command not found
Root Cause	Amazon Linux 2023 does not include wget by default
Fix Applied	Used curl -O instead (available by default)

```
# Fix: Use curl instead of wget
curl -O https://wordpress.org/latest.tar.gz

# Alternative: Install wget
sudo dnf install -y wget
```

Issue 2: HTTP 500 Internal Server Error

Attribute	Detail
Problem	Browser showed 500 Internal Server Error when accessing WordPress
Root Cause	Incorrect file permissions or SELinux blocking Apache access
Diagnostic command	sudo tail -f /var/log/httpd/error_log
Fix Applied	Corrected file ownership and SELinux contexts

```
# Check Apache error log
sudo tail -50 /var/log/httpd/error_log

# Fix file ownership
sudo chown -R apache:apache /var/www/html/wordpress

# Fix SELinux context
sudo chcon -t httpd_sys_rw_content_t /var/www/html/wordpress -R

# Restart Apache
sudo systemctl restart httpd
```

Issue 3: WordPress Could Not Write wp-config.php

Attribute	Detail
Problem	WordPress installer could not automatically create wp-config.php
Root Cause	PHP-FPM running as 'apache' did not have write permission
Fix Applied	Manually copied and edited wp-config.php with correct credentials

```
# Manual fix
sudo cp /var/www/html/wordpress/wp-config-sample.php \
    /var/www/html/wordpress/wp-config.php

sudo nano /var/www/html/wordpress/wp-config.php
# Updated DB_NAME, DB_USER, DB_PASSWORD, DB_HOST
```

Issue 4: MariaDB — Server Fully Qualified Domain Name Warning

Attribute	Detail
Warning message	httpd[24177]: Could not reliably determine the server's fully qualified domain name
Root Cause	Apache could not resolve the hostname to a FQDN (no domain registered, only IP)
Impact	Non-critical — warning only, does not affect functionality
Fix Applied	Added ServerName directive to Apache config

```
# Add to /etc/httpd/conf/httpd.conf or wordpress.conf
ServerName 13.232.39.20

# Test and reload
sudo apachectl configtest
sudo systemctl reload httpd
```

Debugging Commands Reference

These commands were used throughout troubleshooting:

```
# Check Apache error logs (most useful)
sudo tail -100 /var/log/httpd/error_log

# Check Apache access logs
sudo tail -50 /var/log/httpd/access_log

# Check systemd service logs
sudo journalctl -u httpd -n 50
sudo journalctl -u mariadb -n 50
```

```
# Check SELinux denials
sudo ausearch -m avc -ts recent
sudo audit2allow -a

# Check listening ports
sudo ss -tlnp

# Check PHP version and loaded modules
php -v
php -m | grep -i mysql
```

8. Security Hardening

After WordPress was installed and verified, the following security hardening measures were applied:

8.1 Disable Directory Listing

```
# In the virtual host or .htaccess:
Options -Indexes

# This prevents Apache from showing a list of files when no
# index file (index.php, index.html) is found in a directory.
```

8.2 Protect wp-config.php

```
# Add to /var/www/html/wordpress/.htaccess
<Files wp-config.php>
    Order allow,deny
    Deny from all
</Files>
```

Why protect wp-config.php?

wp-config.php contains the database credentials, security keys, and secret salts. If accessible via HTTP, an attacker could view all credentials. The <Files> directive tells Apache to deny all HTTP requests for this specific file.

8.3 Protect .htaccess

```
<Files .htaccess>
    Order allow,deny
    Deny from all
</Files>
```

8.4 Configure Secure File Permissions

```
# Directories: 755 (rwxr-xr-x)
sudo find /var/www/html/wordpress -type d -exec chmod 755 {} \;

# Files: 644 (rw-r--r--)
sudo find /var/www/html/wordpress -type f -exec chmod 644 {} \;

# wp-config.php: 640 (rw-r----- - extra restriction)
sudo chmod 640 /var/www/html/wordpress/wp-config.php
```

8.5 Security Hardening Summary

Hardening Measure	Status
Directory listing disabled (Options -Indexes)	Applied
wp-config.php protected via .htaccess	Applied
.htaccess itself protected	Applied
File permissions set to 644/755	Applied
wp-config.php restricted to 640	Applied
SELinux enforcing mode maintained	Applied
MariaDB secured via mysql_secure_installation	Applied
Database user restricted to localhost only	Applied

9. Backup Procedures

9.1 Backup WordPress Files

```
# Create a timestamped backup of all WordPress files
sudo tar -czvf /backup/wordpress-files-$(date +%Y%m%d).tar.gz \
    /var/www/html/wordpress

# Verify the backup
ls -lh /backup/
```

9.2 Backup the WordPress Database

```
# Create a SQL dump of the wordpressdb database
sudo mysqldump -u root -p wordpressdb > \
    /backup/wordpressdb-$(date +%Y%m%d).sql

# Verify the backup file
ls -lh /backup/
head -5 /backup/wordpressdb-*.sql
```

9.3 Restore Procedure (Reference)

```
# Restore files:
sudo tar -xzf /backup/wordpress-files-YYYYMMDD.tar.gz -C /

# Restore database:
sudo mysql -u root -p wordpressdb < /backup/wordpressdb-YYYYMMDD.sql
```

Backup Best Practices

Always backup both the files AND the database — one without the other is incomplete.
Store backups on a separate location (S3 bucket, external drive) — not on the same server.
Use cron jobs to automate regular backups:
 0 2 * * * /path/to/backup.sh (runs daily at 2 AM)
Test restores periodically to ensure backups are valid.

10. Lab Summary Documentation

10.1 Installed Packages

Package	Version / Notes
httpd (Apache)	2.4.63-13.el10.x86_64
PHP	8.3.29-1.el10_1.x86_64
php-fpm	8.3.29-1.el10_1.x86_64
php-mysqlnd	8.3.29-1.el10_1.x86_64
php-xml, php-mbstring, php-gd	8.3.29-1.el10_1.x86_64
php-pdo, php-opcache, php-intl	8.3.29-1.el10_1.x86_64
php-pecl-zip	1.22.3-5.el10.x86_64
mariadb-server	10.11.15-2.el10_1.x86_64
mariadb (client)	10.11.15-2.el10_1.x86_64
mariadb-backup	10.11.15-2.el10_1.x86_64
WordPress	Latest (downloaded May 2026)

10.2 Service Status

Service	Status
httpd (Apache)	Active (running) — Enabled on boot
mariadb (MariaDB)	Active (running) — Enabled on boot
php-fpm (PHP FastCGI)	Active (running) — Enabled on boot

10.3 Database Details

Parameter	Value
MariaDB Version	10.11.15
Database Name	wordpressdb
Database User	wpuser
Host	localhost
Character Set	utf8mb4
Collation	utf8mb4_unicode_ci
Tables (post-install)	12 WordPress core tables (wp_ prefix)

10.4 Website Access URLs

URL	Purpose
http://13.232.39.20/	WordPress public site
http://13.232.39.20/wp-admin/	WordPress admin dashboard
http://13.232.39.20/wp-login.php	Admin login page

10.5 Important File Paths

Path	Purpose
/var/www/html/wordpress	WordPress root directory
/var/www/html/wordpress/wp-config.php	WordPress configuration file
/etc/httpd/conf.d/wordpress.conf	Apache virtual host configuration
/etc/httpd/conf/httpd.conf	Main Apache configuration
/var/log/httpd/error_log	Apache error log
/var/log/httpd/access_log	Apache access log
/etc/php-fpm.d/www.conf	PHP-FPM pool configuration
/var/lib/mysql/	MariaDB data directory

11. Viva Questions & Answers

The following questions cover key concepts from this lab for exam and viva preparation.

Q1. What is the LAMP stack?

LAMP stands for Linux, Apache, MySQL/MariaDB, and PHP. It is a popular open-source web server stack used to host dynamic websites. Linux provides the OS, Apache serves web requests, MariaDB stores data, and PHP processes server-side logic.

Q2. What is the difference between `systemctl start` and `systemctl enable`?

`systemctl start` launches the service immediately for the current session only. `systemctl enable` configures the service to start automatically every time the system boots. Both are typically used together to ensure the service runs now and after any restart.

Q3. What is SELinux and why does it matter for Apache?

SELinux (Security-Enhanced Linux) is a mandatory access control mechanism. It enforces policies that restrict what processes can do, even if traditional Unix permissions allow it. For Apache, files must have the correct SELinux context (`httpd_sys_content_t` or `httpd_sys_rw_content_t`) or Apache will be denied access, resulting in 403 errors.

Q4. Why is `utf8mb4` used instead of `utf8` for the database?

MySQL's 'utf8' character set only supports 3-byte characters (up to U+FFFF). `utf8mb4` supports the full 4-byte Unicode range, including emojis and characters from less common scripts. WordPress recommends `utf8mb4` to avoid data corruption with such characters.

Q5. What is PHP-FPM and how does it work with Apache?

PHP-FPM (FastCGI Process Manager) is a PHP implementation that uses FastCGI to communicate with Apache. When Apache receives a .php request, it passes it to PHP-FPM via a Unix socket or TCP port. PHP-FPM maintains a pool of worker processes to handle requests efficiently, improving performance over older CGI methods.

Q6. What is `wp-config.php` and why must it be protected?

`wp-config.php` is WordPress's core configuration file. It contains the database name, username, password, hostname, authentication keys/salts, and table prefix. If accessible via HTTP, anyone could steal database credentials. It must be protected using `.htaccess` rules or file permissions (640).

Q7. Why does MySQL need `FLUSH PRIVILEGES` after granting privileges?

`GRANT` statements automatically flush privileges in modern MariaDB. However, if direct modifications are made to the `mysql.user` table (via `INSERT` or `UPDATE`), `FLUSH PRIVILEGES` reloads the grant tables from disk so the changes take effect without restarting MariaDB.

Q8. What does AllowOverride All do in Apache config?

AllowOverride All tells Apache to allow .htaccess files in that directory to override the main server configuration. WordPress uses .htaccess to enable clean URLs (pretty permalinks). Without it, WordPress URLs like /post-name/ would not work.

Q9. What is the difference between curl and wget?

Both are command-line tools for downloading files over HTTP/HTTPS. wget is a standalone downloader with recursive download support. curl is more versatile — it supports more protocols (FTP, SMTP, etc.) and is used in scripting. curl comes pre-installed on Amazon Linux 2023; wget does not.

Q10. What is mysqldump and how is it used?

mysqldump is a command-line utility that exports a MySQL/MariaDB database into a SQL file containing all the CREATE TABLE and INSERT statements needed to recreate the database. It is the standard tool for database backups and is used as: `mysqldump -u root -p database_name > backup.sql`

12. Key Concepts & Glossary

Term	Definition
Apache (httpd)	Open-source HTTP web server software. Handles incoming web requests.
PHP	Hypertext Preprocessor — server-side scripting language used by WordPress.
PHP-FPM	FastCGI Process Manager — runs PHP as a service, pooling worker processes.
MariaDB	Open-source relational database forked from MySQL. Stores WordPress data.
SELinux	Security-Enhanced Linux — mandatory access control built into the kernel.
Virtual Host	Apache configuration that allows one server to host multiple websites.
dnf	Dandified Yum — package manager for Red Hat/Fedora-based Linux distributions.
systemctl	Command to manage systemd services (start, stop, enable, status).
firewalld	Dynamic firewall daemon for managing iptables rules on RHEL systems.
wp-config.php	WordPress main configuration file with DB credentials and security keys.
chown	Change file ownership command. Format: chown user:group /path
chmod	Change file permissions command. Format: chmod 755 /path
chcon	Change SELinux security context of files.
mysqldump	Command-line tool to export MySQL/MariaDB databases to a SQL file.
AllowOverride	Apache directive enabling .htaccess to override server configuration.
DocumentRoot	Apache directive specifying the directory from which content is served.
curl	Command-line tool for data transfer using various network protocols.
tar	Archive utility for creating and extracting .tar.gz compressed archives.
RPM	Red Hat Package Manager — package format used on RHEL-based systems.
LAMP Stack	Linux + Apache + MySQL/MariaDB + PHP — a common web hosting stack.

